



More named universal character escapes

Document number:

[P3733R1](#)

Date:

2025-09-27

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:

Jan Schultke <janschultke@gmail.com>

GitHub Issue:

wg21.link/P3733/github

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/more-unicode-escapes.cow



ABSTRACT: C++23 permits the use of "correction", "control", and "alternate" aliases for character names, but not "figment" or "abbreviation". Following P2736R2, this restriction is no longer necessary because "figment" and "abbreviation" are normatively specified in the Unicode standard.

§ Contents

- 1 **Revision history**
 - 1.1 Changes since R0
- 2 **Introduction**
 - 2.1 History
 - 2.2 Inconsistency with other languages
- 3 **Motivation**
 - 3.1 Abbreviations
 - 3.2 Figments
 - 3.3 Category stability guarantees
- 4 **Proposed change**

5	Impact on implementations
6	Wording
7	References



§ 1. Revision history

§ 1.1. Changes since R0

The paper was seen by SG16 and discussed, with the following poll results:

”

Poll 1: P3733R0: Forward to EWG with a recommendation to adopt this feature as a defect report.

- Attendees: 10

•	SF	F	N	A	SA
	7	3	0	0	0

- Unanimous consensus.

The following changes were made:

- Added [§3.3. Category stability guarantees](#)
- Provided rationale for adopting this feature as a defect report in [§4. Proposed change](#)
- Rebased [§6. Wording](#) on [\[N5014\]](#) (no changes needed)

§ 2. Introduction

§ 2.1. History

[\[P2071R2\]](#) introduced *named-universal-character* escapes into C++23, which makes it possible to use "escape sequences" like `\N{NO-BREAK SPACE}` within string or character literals. These provide much-needed clarity as compared to `\u00A0`. Some code points additionally or exclusively have aliases. For example, DELETE (control alias) and DEL (abbreviation alias) correspond to U+007F within the Unicode standard. There is no name for U+007F that is not categorized as an alias.

SG16 voted unanimously to support aliases within a *named-universal-character* at Prague 2020 when discussing R0:

”

Match name aliases?

SF	F	N	A	SA
8	2	0	0	0



EWG reaffirmed that decision at the same meeting:

”

This [named universal character escapes] should further support aliases

SF	F	N	A	SA
18	2	1	0	0

However, *some* categories of aliases are disallowed in C++, as explained in [\[P2071R2\] §8.2 Name sources](#):

”

Unicode aliases provide another critical service. As mentioned above, once assigned, names are immutable. Corrections are only offered by providing an alias. Aliases, according to the NamedAliases tables in the Unicode Character Database, come in five varieties:

- **correction** Aliases for cases where an incorrect assigned name was published. For example, U+FE18 has an assigned name of PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRACKET and a correction alias of PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRACKET (note the typo correction).
- **control** Aliases for various control characters. For example, NULL for U+0000.
- **alternate** Aliases for widely used alternate names. For example, BYTE ORDER MARK for U+FEFF.
- **figment** Aliases for names that were documented, but never accepted in a standard. For example, HIGH OCTET PRESET for U+0081.
- **abbreviation** Aliases for common abbreviations. For example, NBSP for U+00A0.

The intent is to use the aliases classified as correction, control, and alternate as recognized names.

While the paper does not make it obvious why figment and abbreviation are excluded, the underlying reason is that the C++ standard referenced ISO/IEC 10646 at the time, where figment aliases are not included whatsoever, and where only a subset of the abbreviation aliases in the Unicode standard is included. These issues were discussed at the 2021-11-03 SG16 meeting for R1; see [\[P2512R0\]](#). R2 was then plenary-approved in 2022 with no support for abbreviation and figment aliases.

Following [\[P2736R2\]](#), the C++ standard references the Unicode standard instead of ISO/IEC 10646, and such a restriction is no longer motivated.

§ 2.2. Inconsistency with other languages

Many design choices of [P2071R2] are ultimately motivated by [P2071R2] §8.5 Existing practice. For example, the `\N{...}` syntax in C++ is identical to Python and Perl. While C++ shares a syntax, it does not permit the same categories of aliases:

Alias category	Example	C++	Python	Perl
correction	<code>\N{PRESENTATION FORM FOR VERTICAL RIGHT WHITE LENTICULAR BRACKET}</code>	✓	✓	✓
control	<code>\N{NULL}</code>	✓	✓	✓
alternate	<code>\N{BYTE ORDER MARK}</code>	✓	✓	✓
figment	<code>\N{HIGH OCTET PRESET}</code>	✗	✓	✓
abbreviation	<code>\N{NBSP}</code>	✗	✓	✓

Although this made historical sense, it now feels like an arbitrary restriction.

§ 3. Motivation

Besides the inconsistency with other languages above, there are good reasons to allow abbreviations and figments in C++ (or more generally, allow *any* category of alias).

§ 3.1. Abbreviations

While the usefulness of some abbreviations is debatable, some of them significantly shorten commonly used code points.



EXAMPLE: Multi-part emoji are constructed using U+200D ZERO WIDTH JOINER, which is a rather long name:

```
// Without abbreviations, we can form a "family: woman, woman, girl" 🧑👩👧
u8"\N{WOMAN}\N{ZERO WIDTH JOINER}\N{WOMAN}\N{ZERO WIDTH JOINER}\N{GIRL}"
// With abbreviations:
u8"\N{WOMAN}\N{ZWJ}\N{WOMAN}\N{ZWJ}\N{GIRL}"
```



EXAMPLE: If we log messages into a UTF-8 text file, it is quite plausible that we would occasionally want to use U+00A0 NO-BREAK SPACE or U+00AD SOFT HYPHEN code points:

```
// Without abbreviations:
u8"INFO: Auto\N{SOFT HYPHEN}reconnect triggered due to network\N{NO-BREAK SPACE}timeout."
// With abbreviations:
u8"INFO: Auto\N{SHY}reconnect triggered due to network\N{NBSP}timeout."
```

All that is to say that some abbreviations are well-motivated.

§ 3.2. Figments



There are currently only three aliases classified as figment:

- U+0080 PADDING CHARACTER
- U+0081 HIGH OCTET PRESET
- U+0099 SINGLE GRAPHIC CHARACTER INTRODUCER

While there also exist PAD, HOP, and SGC abbreviations for these characters, these are rather obscure and a user may prefer to use the figment names for additional clarity. Therefore, they should also be supported by C++.



NOTE: An alias being considered a figment is inconsequential. See [\[UnicodeNameAliases\]](#) for an explanation as to why these names are listed despite never being standardized "intentionally".

§ 3.3. Category stability guarantees

While the Unicode standard guarantees that aliases are not changed or removed, it does not have a stability guarantee for the alias categories. It is theoretically possible for the figment category to be merged into alternate, for example, although this seems implausible and unmotivated at this time.

In any case, simply permitting *any* category of alias removes a reliance on an unstable property of the Unicode standard.

§ 4. Proposed change

The abbreviation and figment categories should also be permitted within a *named-universal-character*.

As recommended by SG16, this change should be applied as a defect report. It would be quite inconvenient to the user if they had to refrain from using `\N{NBSP}` if they wanted their code to be compatible with C++23. Since the motivation for disallowing that name disappeared with [\[P2736R2\]](#) (still in C++23), the proposed change is arguably fixing a defect in the language.

§ 5. Impact on implementations

Permitting abbreviations and figments is essentially trivial. [\[UnicodeNameAliases\]](#) contains a list of all aliases, with 354 abbreviations and 3 figments. This is a drop in the ocean compared to the existing set of names.

Furthermore, the same guarantees of uniqueness (will never conflict with other names) and immutability (will never change) are provided for figment and abbreviation as for the

control, alternate, and correction categories. This is generally the case for any names listed in [UnicodeNameAliases]. See [UnicodeAliasStability]. Therefore, upwards compatibility is not threatened.



§ 6. Wording

The following change is relative to [N5014].

Change [lex.universal.char] paragraph 3 as follows:



A *universal-character-name* that is a *named-universal-character* designates the corresponding character in the Unicode Standard (chapter 4.8 Name) if the *n-char-sequence* is equal to its character name or to one of its character name aliases ~~of type “control”, “correction”, or “alternate”~~; otherwise, the program is ill-formed.

[Note: These aliases are listed in the Unicode Character Database's NameAliases.txt. None of these names or aliases have leading or trailing spaces. — end note]

Change a feature-test macro in [tab.cpp.predefined.ft] as follows:



Macro name	Value
[...]	[...]
__cpp_named_character_escapes	202207L 20XXXXL
[...]	[...]

§ 7. References

[N5014] Thomas Köppe. Working Draft, Programming Languages — C++ (2025-08-05)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5014.pdf>

[P2071R2] Tom Honermann et al.. Named universal character escapes (2022-03-25)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2071r2.html>

[P2512R0] Tom Honermann. SG16: Unicode meeting summaries 2021-06-09 through 2021-12-15 (2021-12-23)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2512r0.html>

[P2736R2] Corentin Jabot. Referencing The Unicode Standard (2023-02-09)
<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2736r2.pdf>

[UnicodeNameAliases] Unicode® NameAliases.txt
<https://www.unicode.org/Public/16.0.0/ucd/NameAliases.txt>

[UnicodeAliasStability] Unicode® Character Encoding Stability Policies — Formal Name Alias Stability
https://www.unicode.org/policies/stability_policy.html#Formal_Name_Alias